# Sequences, Containers, Objects, & Linked Lists Solutions

## Computer Science Mentors 61A

September 28, 2026 – October 2, 2026

## 1 Sequences

### Sequence Overview

Sequences are ordered data structures that have length and support element selection. Here are some common types of sequences you'll be dealing with in this class:

- Lists: `[1, [2], 'a', lambda x: 5]`
- Tuples: `(1, (2,), 'a', lambda x: 5)`
- Strings: `'Hello World!'`

While each type of sequence is different, they all share a common interface for manipulating and accessing their data:

- **Item selection**: Use square brackets to select an element at an index:

```
(3, 1, 2)[0]   # returns 3
"Hello"[-1]    # returns "o"
```

- **Length**: The built-in `len` function returns the length of a sequence:

```
len((1, 2))   # returns 2
```

- **Concatenation**: Sequences can be concatenated with the + operator, which returns a *new* sequence:

```
[1, 2] + [3, 4]   # returns [1, 2, 3, 4]
```

- **Membership**: The `in` operator tests for sequence membership:

```
1 in (1, 2, 3)           # returns True
5 not in (1, 2, 3)       # returns True
"apple" in "snapple"     # returns True
```

  **Membership in Strings vs. Lists and Tuples**: As a short aside, while the `in` operator works the same for lists and tuples, checking if an element is contained within the list/tuple container, the `in` operator instead for strings checks for direct substrings rather than the existence of distinct elements within the string.

- **Looping**: Sequences can be looped through with `for` loops:

```
>>> for x in [1, 2, 3]:
...     print(x)
1
2
3
```

Created by Cadence Loh and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

- **Aggregation**: Common built-in functions—including **`sum`**, **`min`**, and **`max`**—can take sequences and aggregate them into a single value:

  ```
  max((3, 4, 5))  # returns 5
  ```

- **Slicing**: Slicing is a way to create a copy of all or part of a sequence. The general syntax for slicing a sequence `seq` is as follows:

  ```
  seq[<start index>:<end index>:<step size>]
  ```

  This evaluates to a new sequence that includes every element starting at `<start index>` and up to and *excluding* `<end index>` in `seq`, taking steps of size `<step size>`.
  If we do not supply `<start index>` or `<end index>`, it will start at the beginning of the sequence and include every element up to and including the end of the sequence.

  ```
  >>> lst = [1, 2, 3, 4, 5]
  >>> lst[2:]
  [3, 4, 5]
  >>> lst[:3]
  [1, 2, 3]
  >>> lst[::-1]
  [5, 4, 3, 2, 1]
  >>> lst[1::2]
  [2, 4]
  ```

**List comprehensions** create a new list by processing an iterable, such as a list, tuple, string, or range. The syntax for a list comprehension is

```
[<expression> for <element> in <sequence> if <condition>]
```

We could equivalently write the following:

```
lst = []
for <element> in <sequence>:
    if <condition>:
        lst = lst + [<expression>]
```

The **`if`** `<condition>` filter statement is optional. The following list comprehension doubles each odd element of [1, 2, 3, 4]:

```
>>> [i * 2 for i in [1, 2, 3, 4] if i % 2 != 0]
[2, 6]
```

Equivalent in **`for`** loop syntax:

```
lst = []
for i in [1, 2, 3, 4]:
    if i % 2 != 0:
        lst = lst + [i * 2]
```

## Sequence Practice

1. What would Python display? Run the statements in order. For nested expressions, work from the innermost expression outward.

```
>>> a = [1, 2, 3]
>>> a

[1, 2, 3]

>>> a[2]

3

>>> a[-1]

3

>>> a[1:]

[2, 3]

>>> a[:2]

[1, 2]

>>> a + [4, 5]

[1, 2, 3, 4, 5]

>>> a

[1, 2, 3]

>>> nested = [1, 2, 3, 4, [5, 6]]
>>> nested[4]

[5, 6]
```

```
>>> nested[4][0]

5

>>> nested[3:5]

[4, [5, 6]]
```

2. **Tuples.** What would Python display? Run these statements in order. If a statement causes an error, identify the error and continue with the next statement. For `(3,)` and `(3)`, also name the type.

   **Creating tuples**

```
>>> t: tuple[int, str, float, bool, None] = (
...     3, "csm", 2.5, True, None
... )
>>> t

(3, 'csm', 2.5, True, None)

>>> 4, 5, 6, 7

(4, 5, 6, 7)

>>> (3,)

(3,)
```

   Type: **`tuple`**.

```
>>> (3)

3
```

   Type: **`int`**.

```
>>> ()
>>> len(())

()
0
```

### Slicing and converting

```
>>> t[1:4]

('csm', 2.5, True)

>>> t[::-1]

(None, True, 2.5, 'csm', 3)

>>> list(t)

[3, 'csm', 2.5, True, None]

>>> list("csm")
>>> list(())

['c', 's', 'm']
[]
```

### Combining tuples

```
>>> (4, 5) + (6, 7)

(4, 5, 6, 7)

>>> (3,) * 3
>>> (3) * 3

(3, 3, 3)
9
```

### Dictionary keys

```
>>> locations = {(4, 5): "CSM"}
>>> locations[(4, 5)]

'CSM'

>>> locations[[4, 5]]

TypeError
```

```
>>> locations[(4, [5])]
```

```
TypeError
```

3. Write a function `duplicate_list`, which takes in a list of positive integers and returns a new list with each element `x` in the original list duplicated `x` times.

```
def duplicate_list(lst: list[int]) -> list[int]:
    """
    >>> duplicate_list([1, 2, 3])
    [1, 2, 2, 3, 3, 3]
    >>> duplicate_list([5])
    [5, 5, 5, 5, 5]
    """
    ____________________________________

    for ____________________________________:

        for ____________________________________:

            ____________________________________

    ____________________________________
```

```
new_list = []
for x in lst:
     for i in range(x):
          new_list = new_list + [x]
return new_list
```

4. Write a function that takes in a list `nums` and returns a new list with only the primes from `nums`. Assume that `is_prime(n)` is defined. You may use a **`while`** loop, a **`for`** loop, or a list comprehension.

```
def all_primes(nums: list[int]) -> list[int]:


    result = []
    for i in nums:
        if is_prime(i):
            result = result + [i]
    return result

    # Alternative using a list comprehension:
    return [x for x in nums if is_prime(x)]
```

5. Fill in the functions below according to the doctests.

```
def gen_list(n: int) -> list[list[int]]:
    """
    Returns a nested list structure of n elements where the
    ith element is a list from 0 (inclusive) to i (exclusive).
    >>> gen_list(3)
    [[0], [0, 1], [0, 1, 2]]
    >>> gen_list(5)
    [[0], [0, 1], [0, 1, 2], [0, 1, 2, 3], [0, 1, 2, 3, 4]]
    """
    return _____________________________________________________
```

For an additional challenge, try out the following:

```
def gen_increasing(n: int) -> list[list[int]]:
    """
    Returns a nested list structure of n elements where the
    ith element of each list is one more than the previous
    element (even if the previous is in a prior sublist).
    >>> gen_increasing(3)
    [[0], [1, 2], [3, 4, 5]]
    >>> gen_increasing(5)
    [[0], [1, 2], [3, 4, 5], [6, 7, 8, 9], [10, 11, 12, 13,
    14]]
    """
    return _____________________________________________________
```

**Hint:** You can sum ranges. E.g. `sum(range(3))` gives us $0 + 1 + 2 = 3$.

```
def gen_list(n: int) -> list[list[int]]:
    return [[i for i in range(j+1)] for j in range(n)]

def gen_increasing(n: int) -> list[list[int]]:
    return [[i for i in range(sum(range(j+1)), sum(range(j+1)) + j+1)] for
        j in range(n)]
```

*An alternate solution for `gen_increasing` is shown below:*

```
def gen_increasing(n: int) -> list[list[int]]:
    return [[i + sum(range(j + 1)) for i in range(j + 1)] for j in
        range(n)]
```

6. Write `count_t`, which returns the number of lowercase `"t"` characters in `word`. Then use it to construct a dictionary mapping each word in `words` to its count.

```
def count_t(word: str) -> int:
    """
    >>> count_t("tatter")
    3
    >>> count_t("tree")
    1
    >>> count_t("")
    0
    """
    ____________________________________

    for ____________________________________:

        if ____________________________________:

            ____________________________________

    ____________________________________
```

```
def count_t(word: str) -> int:
    count = 0
    for c in word:
        if c == 't':
            count += 1
    return count
```

Complete the dictionary comprehension. Its result should be `{'tatter': 3, 'tree': 1}`.

```
words = ["tatter", "tree"]
counts: dict[str, int] = {
    _____________: _____________ for word in words
}
```

```
counts: dict[str, int] = {
    word: count_t(word) for word in words
}
```

7. Complete `snapshot`. It takes a function from integers to integers and a list of integer inputs. Return a dictionary whose keys are those inputs and whose values are the corresponding outputs of the function. Use a dictionary comprehension.

```
from collections.abc import Callable

def snapshot(
    f: Callable[[int], int], snap_inputs: list[int]
) -> dict[int, int]:
    """
    >>> snapshot(lambda x: x**2, [1, 2, 3])
    {1: 1, 2: 4, 3: 9}
    >>> snapshot(lambda x: x + 1, [])
    {}
    """
    return {
        _____________: _____________
        for ___________________________________
    }
```

```
def snapshot(
    f: Callable[[int], int], snap_inputs: list[int]
) -> dict[int, int]:
    return {x: f(x) for x in snap_inputs}
```

8. (a) What is an **Abstract Data Type**?

   An abstract data type describes the information a value represents and the operations we can use with it, without requiring users to know how those operations are implemented. For example, a session record can provide a topic, room, and duration through named attributes.

   (b) What is the relationship between a class and an ADT?

   A class is one way to implement an ADT. A dataclass can bundle a session's information into fields such as `topic`, `room`, and `minutes`. Calling `Session("Recursion", 310)` constructs an instance, and reading `s.room` accesses its room. Code using this interface need not know how the fields are stored. The class's definition implements the representation; the meaning and supported operations form the abstraction.

9. **Session records.** What would Python display after executing the following definitions? Read the field annotations before tracing.

```
from dataclasses import dataclass

@dataclass
class Session:
    topic: str
    room: int
    minutes: int = 60

morning = Session("Recursion", 310)
afternoon = Session("Sequences", 205, 45)
```

```
>>> morning.topic
'Recursion'
>>> morning.minutes
60
>>> afternoon.room
205
>>> afternoon.minutes
45
```

```
>>> morning.minutes + afternoon.minutes
105
>>> morning
Session(topic='Recursion', room=310, minutes=60)
>>> Session(room=220, topic="Linked lists").topic
'Linked lists'
```

# 4 Linked Lists

For the following questions, use this dataclass-based linked list. A `LinkedList[T]` is either a `Link[T]` or the empty tuple `()`. The type variable `T` describes the element type.

```
from __future__ import annotations
from dataclasses import dataclass

type LinkedList[T] = Link[T] | tuple[()]

@dataclass
class Link[T]:
    first: T
    rest: LinkedList[T] = ()

    def __str__(self) -> str:
        return format_link(self)

def format_link[T](s: Link[T]) -> str:
    string = '(' + str(s.first)
    remaining = s.rest
    while isinstance(remaining, Link):
        string += ' ' + str(remaining.first)
        remaining = remaining.rest
    assert remaining == (), 'Not a valid linked list'
    return string + ')'
```

Construct a list with `Link(1, Link(2))`. Read its elements with `first` and `rest`. At the end, `rest` is `()`, which has no `first` or `rest` attributes. **`print`** uses parentheses around the elements; entering a `Link` at the prompt shows the dataclass representation with field names.

10. What would Python display? Run these statements in order. If a statement causes an error, identify the error and continue.

```
>>> a = Link(1, Link(2, Link(3)))
>>> a.first
1
>>> a.rest.first
2
>>> a.rest.rest.first
3
```

```
>>> a.rest.rest.rest
()
>>> isinstance(a.rest, Link)
True
>>> isinstance(a.rest.rest.rest, Link)
False
>>> a.rest.rest.rest.first
AttributeError: 'tuple' object has no attribute 'first'
>>> a.rest.rest
Link(first=3, rest=())
>>> Link(1, Link(2))
Link(first=1, rest=Link(first=2, rest=()))
>>> print(a)
(1 2 3)
```

11. Write a function `linkify` that, given a list `lst`, returns a linked list with the same values. You may assume that there will be no nested lists.

    Try writing this function both recursively and iteratively.

```
def linkify_rec[T](lst: list[T]) -> LinkedList[T]:
    """
    >>> lst = [0, 1, 2, 3]
    >>> print(linkify_rec(lst))
    (0 1 2 3)
    >>> linkify_rec([])
    ()
    """
    if ____________________________________:

        ____________________________________

    else:

        ____________________________________


def linkify_iter[T](lst: list[T]) -> LinkedList[T]:

    ____________________________________

    for ____________________________________:

        ____________________________________

    ____________________________________
```

```
def linkify_rec[T](lst: list[T]) -> LinkedList[T]:
    if not lst:
        return ()
    else:
        return Link(lst[0], linkify_rec(lst[1:]))

def linkify_iter[T](lst: list[T]) -> LinkedList[T]:
    retVal: LinkedList[T] = ()
    for elem in lst[::-1]:
        retVal = Link(elem, retVal)
    return retVal
```

12. Write a function `combine_two`, which takes in a linked list of integers `lnk` and a two-argument function `fn`. It returns a new linked list where every two elements of `lnk` have been combined using `fn`.

```
from collections.abc import Callable
def combine_two(
    lnk: LinkedList[int], fn: Callable[[int, int], int]
) -> LinkedList[int]:
    """
    >>> from operator import add, mul
    >>> lnk1 = Link(1, Link(2, Link(3, Link(4))))
    >>> print(combine_two(lnk1, add))
    (3 7)
    >>> lnk2 = Link(2, Link(4, Link(6)))
    >>> print(combine_two(lnk2, mul))
    (8 6)
    >>> combine_two((), add)
    ()
    """
    if ____________________________________________:

        return ____________________________________
    elif ____________________________________________:

        return ____________________________________
    combined = ____________________________________

    return ____________________________________________


def combine_two(
    lnk: LinkedList[int], fn: Callable[[int, int], int]
) -> LinkedList[int]:
    if not isinstance(lnk, Link):
        return ()
    elif not isinstance(lnk.rest, Link):
        return Link(lnk.first)
    combined = fn(lnk.first, lnk.rest.first)
    return Link(combined, combine_two(lnk.rest.rest, fn))
```

## 5 Optional Extension: Tree-Recursive Counting

13. Gabe's Donut shop has an unlimited supply of $f$ different flavors of donuts. Adit wants to buy a box containing $d$ donuts. Complete the skeleton for the function `donut`, which determines the number of possible ways there are for Adit to select his $d$ donuts from the $f$ flavors. You may assume that `d` and `f` are non-negative integers.

*Hint: Does order matter?*

```
def donut(d: int, f: int) -> int:
    """
    >>> donut(12, 1)
    1
    >>> donut(12, 2)
    13
    >>> donut(12, 12)
    1352078
    >>> donut(0, 0)
    1
    """
    if _____________________:

        return _____________________

    if _____________________:

        return _____________________

    return ____________________________________________
```

```
def donut(d: int, f: int) -> int:
    if d == 0:
        return 1
    if f == 0:
        return 0
    return donut(d - 1, f) + donut(d, f - 1)
```

Suppose the flavors are numbered 1 through $f$. Because order does not matter, we're going to say that Adit fills up his donut box starting with flavor number $f$ and working down to flavor number 1. Under this scheme, Adit has a choice to make: does he want a donut of flavor number $f$, or does he not want one? If he does initially take a donut of flavor number $f$, then the number of ways to select the remaining $d-1$ slots of the box is `donut(d - 1, f)`. If he does not want a donut of flavor number $f$, then he has $d$ slots left in the box and $f-1$ flavors to fill them with, making the number of ways to make the selection `donut(d, f - 1)`. Since he must make the choice one way or the other, the total number of ways to fill up the box is `donut(d - 1, f)+ donut(d, f - 1)`.